Graph Analytics Orchestrator: Course Project Report

Nov 5, 2026 · Gowni Vamshi

CS6855 course project, Group [number]. Our orchestrator answers natural-language questions about the course graphs by planning and chaining registry tools; it scores 30/30 on the public query set.

1. Design

The orchestrator is a plain-Python agent loop (no agent framework) over an OpenAI-compatible API. It takes a question through five stages, and every fact in the final answer must come from a logged tool result.

1. **Tool discovery.** At startup we read every `tool.json` under `infra_tools/` and `registry/tools/` and turn each manifest into a function schema; no tool name is hard-coded. 37 tools load: 33 curated menu tools and the 4 instructor tools.
2. **Triage.** A small LLM step first decides only whether the question can be answered as written: proceed, clarification_needed or unsupported. It sees the graph catalogue and one line per tool, not the full schemas. Two judges, Groq's `openai/gpt-oss-120b` and Gemini's `gemini-3.5-flash-lite`, must reach the same stop decision; otherwise the question goes to planning.
3. **Planning and execution.** The planner (`gemini-3.5-flash-lite`) gets a prompt built from the graph store (descriptions, direction, weights, attributes, example ids) and all tool schemas plus `final_answer`. Each question runs on a fresh `GraphStore`. Multi-step questions chain through `result_graph_id`: community, core, clique, ego, k-hop and filter tools register subgraphs whose ids the next tool takes as `graph_id`.
4. **Grounded finishing.** Before accepting `final_answer` we check it against the trace: node ids must be real ids seen in a tool output, numbers must appear in one (or be confirmed as computed), and `answered` needs a successful tool call. Rejected answers go back to the model with the reason.
5. **Traces.** `traces/<query_id>.jsonl` logs the question, every LLM call (both triage judges included), every tool call with arguments and output, and the final answer.

Tool curation

All 186 class implementations pass the contract validator, so it cannot choose between them. `curation/curate.py` runs each one on the five datasets, compares outputs with NetworkX, and ranks by score, then determinism, speed and options. 35 scored below 0.9,

for example a motif counter reporting 52 karate triangles instead of 45. Two curation runs chose the same 33 winners; we edited no registry tool.

Conventions and reliability

- **Unweighted centralities**, as in the reference answers: for tools whose manifest category is `centrality`, the executor sets `weighted` / `use_weights` to false unless the planner passes it.
- **Budgets**: at most 6 tool calls and 10 LLM calls per question; then only `final_answer` is offered.
- **Quotas**: responses are cached on disk; rate limits are retried; an exhausted daily quota stops the run cleanly or switches to a backup model. The planner's 15k-token calls stay on Gemini because they exceed Groq's free per-minute limit.

2. Results on the public set

The submitted run answers all 30 public questions correctly. Scoring follows the handout: numbers within 0.1%, rankings in order, other node lists as sets, the course order by checking that every prerequisite comes first, and ambiguous or unsupported questions by status.

Question type	Questions	Correct
Single-tool	q01–q14	14/14
Multi-step	q15–q23	9/9
Ambiguous	q24–q26	3/3
Unsupported	q27–q30	4/4
Total	30	**30/30 (100%)**

The run uses 1.6 tool calls per question on average and at most 4 (the guideline is about five), and 3.6 LLM calls including triage; the seven ambiguous and unsupported questions are settled by triage without tool calls.

Across development the score went from 27/30 (first run) to 29/30 (fixes from section 3) to 30/30 (two-judge triage); the same code with `--no-triage` scores 29/30. Before any LLM run we confirmed that the curated tools reproduce all 23 answerable expected answers when called with the right arguments, so the remaining risk lies in planning, not in the tools.

3. Failure analysis: what broke during development and how we fixed it

Five failures came up during development, each with a different root cause: a model limit, a tool option, an API timeout, an answer type and a data convention. All are fixed in the submitted run.

3.1 Answering an ambiguous question (q26, runs 1 and 3)

“How close are Marius and Cosette?” needs clarification_needed: “close” can mean hops, shared chapters or similarity. The planner ran a weighted shortest path and answered 2: with 37 tool schemas in view, the small model goes straight to tool choice, and prompt rules only moved the error. A flash-lite triage fixed q26 but flagged the clear q06 (“most relevant to Chennai”); gpt-oss-120b alone flagged q15, q18 and q19 over technical variants. Their false alarms did not overlap, so requiring both judges to agree removed them all while both still caught q24–q30.

3.2 Dropping the centre of an ego network (q22, run 1)

For “triangles in Javert’s immediate circle” the planner called `ego_network` with `include_ego=false`, removing Javert and his triangles: 52 instead of 96. The tool was right; the planner chose an option nobody asked for. The prompt now says to build ego networks with the tool’s defaults and to leave the person out only of the answer when asked (q17, q21).

3.3 An API timeout reported as an error (q30, run 1)

The planarity question returned error instead of unsupported. A Gemini request timed out, and our retry list matched “timeout” but not the SDK’s “timed out”. We now retry by exception class as well as by message, and stop cleanly with a message when a daily quota is gone.

3.4 A path where a count was asked (q03, run 2)

A new prompt rule about routes made “the fewest number of flights from Chennai to Lisbon” return MAA → DXB → LIS instead of 2. We added that a count question wants the number even when the tool also returns the path, and we re-run the whole set after every prompt change to catch such regressions.

3.5 Weighted centralities against an unweighted reference (q05, q06, caught before run 1)

The expected answers use unweighted centralities, but our chosen `pagerank` and `personalized_pagerank` weight edges by distance by default, which reorders airports. Calling each expected chain directly against the tools exposed this before any LLM run; the executor now defaults weights off for centrality tools, read from the manifests.

Remaining risk: with only a Gemini key, triage has one judge (flash-lite), which made one wrong decision across our 59 test questions.

4. Member contributions

We are a group of four. Each member led one area, built on the handout's suggested roles, and everyone wrote code. [Group: fill in names, and edit each description to match what that person actually did.]

Role	Member	Led on
Planner and Executor (orchestrator core)	Gowni Vamshi	Agent loop, system and triage prompts, the two-judge triage design, chaining through `result_graph_id`, unweighted-centrality convention, grounding checks, error and quota handling
Tool curator	[name]	`curation/curate.py`: testing all 186 implementations against NetworkX and installing the 33 winners
Evaluation lead	[name]	Public-set conversion, `evaluate.py`, the runs in section 2 and the failure analysis in section 3
Integration and demo lead	[name]	`run_queries.py`, README, clean-install tests and the submission zip

5. Use of AI coding tools

We used Claude Code, an AI coding assistant from Anthropic, to help write and test the code and to draft this report; results and analyses were checked against our own runs and traces. Every member has reviewed the code and can explain it. [Group: confirm this, and add any other tools you used.] At run time the orchestrator uses only free-tier LLMs from Google Gemini and Groq.